HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

School of Information and Communication Technology

PROJECT REPORT

ConfigStream

Version-aware configuration history, durable storage, and reproducible analytics

Checkpoint status through 25 October 2026

Supervisor: PhD Tran Viet Trung

Member
Nguyen Duc Dung
Pham Cong Hoang
Mai Vu Hoang
Le Tien Hop

Ha Noi, 2026

Contents

List of Abbreviations **iv**

1 Checkpoint scope and status **1**

2 Problem and research questions **1**

3 System design **2**

3.1 Architecture at a glance 2

3.2 State and pairing policy 3

3.3 Common evidence contract 3

3.4 Configuration transformation 4

3.5 Source-specific temporal lanes 4

3.6 Publication and recovery 5

3.7 Evaluation method 5

3.8 Normalization, diff, and publication 5

4 Current experimental evidence **6**

4.1 Evidence roles and workload semantics 6

4.2 End-to-end TerraDS replay 6

4.3 Corrected matched processing campaign 6

4.4 Storage and resource observations 7

5 What is complete and what remains **7**

5.1 Completed at the checkpoint 7

5.2 Not yet complete 8

6 Risk assessment **8**

7 Next steps **8**

8 Conclusion **9**

List of Tables

1 Checkpoint status by work package. 1
2 Accepted workload evidence at the checkpoint. 6
3 Matched processing medians and observed ranges. 7
4 Main risks and mitigation actions. 8
5 Milestones after the 25 October 2026 evidence boundary. 9

List of Figures

1 ConfigStream evidence flow. Source-specific gates create typed temporal records, the manifest defines the authoritative commit, and downstream views remain rebuildable. 2

List of Abbreviations

API	Application Programming Interface
DNS	Domain Name System
HA	High Availability
IaC	Infrastructure as Code
JSONL	JSON Lines
RPKI	Resource Public Key Infrastructure
RIS	Routing Information Service
SUT	System Under Test

1 Checkpoint scope and status

This report records the state of ConfigStream through 25 October 2026. It reports implemented components, accepted evidence, pending experiments, and claims that remain outside the tested scope. The date defines the evidence boundary; the report does not convert project progress into a percentage.

The central question is whether a streaming system can preserve enough identity, event-time information, provenance, and validation evidence to support trustworthy configuration-history analysis. ConfigStream addresses this question by combining Kafka ingestion, stateful Spark Structured Streaming, authoritative Parquet history in MinIO, rebuildable search projections, and a read-only API/dashboard. The design makes a deliberate separation between the historical record and the indexes used to browse it.

By this date, the implementation path, the main lake-first publication path, and several correctness and replay artifacts are available. Remaining work concerns comparative evaluation, failure and recovery coverage, reproducibility packaging, and final alignment between evidence and claims.

Table 1: Checkpoint status by work package.

Work package	Status	Evidence boundary
Data contracts and workload generation	Complete	Versioned asset/snapshot/event contracts and domain-specific workload artifacts are present.
Streaming ingestion and state	Complete for current scope	Kafka-to-Spark processing, event-time buffering, deduplication, adjacent-version pairing, and diagnostics are implemented.
Historical storage and projections	Complete for current scope	MinIO/Parquet is treated as authoritative; Elasticsearch and the API are rebuildable read paths.
Dashboard and evidence views	Accepted	The Phase 20V read-only views expose history, aggregates, experiments, and evidence details.
Correctness and replay evaluation	Substantially complete	TerraDS replay and the corrected 27-trial matched campaign have accepted artifacts.
Production-scale and HA evaluation	Pending	Full paced RIS capacity, H3, multi-node HA, and end-to-end exactly-once evidence remain outside the current claim.

2 Problem and research questions

Configuration history is easy to store incorrectly. A system that compares the previous message with the current message can silently construct the wrong transition when messages arrive out of order. A search index can make a result look available while the authoritative history is incomplete. A throughput number can also be misleading when one workload reports rows per second and another reports occurrences, files, or bytes.

ConfigStream therefore treats a configuration change as a provenance-bearing temporal record. A change is meaningful only when the system can identify the asset, order versions using the pro-

ducer's version semantics, reconstruct adjacent states, and retain enough evidence to reproduce the resulting diff or alert.

The checkpoint evaluates three research questions:

1. **RQ1:** Can ConfigStream reconstruct adjacent configuration states correctly under out-of-order arrival, duplicates, bounded buffering, and restart-sensitive streaming execution?
2. **RQ2:** Does a lake-first storage boundary preserve a trustworthy historical record while allowing search and API projections to be rebuilt independently?
3. **RQ3:** How should correctness, throughput, latency, storage growth, and resource use be measured across workloads whose units and semantics are different?

The current evidence answers the implementation portion of these questions positively for the tested scope. It does not yet support claims about cluster-wide scalability, HA, or universal configuration parsing.

3 System design

3.1 Architecture at a glance

Figure 1 summarizes the architecture in the form used for the paper. The figure separates four ideas that are often mixed together in an engineering diagram: the typed record entering the system, the event-time state transition, the publication boundary, and the evidence-aware evaluation loop.

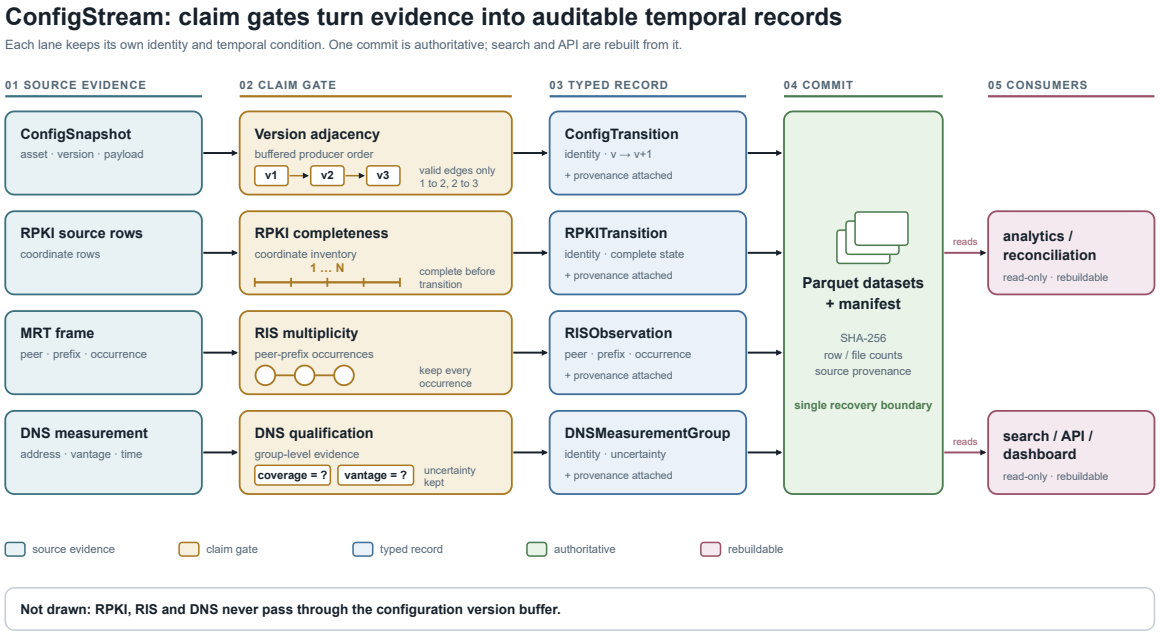


Figure 1: ConfigStream evidence flow. Source-specific gates create typed temporal records, the manifest defines the authoritative commit, and downstream views remain rebuildable.

The implemented path is:

1. a source or generator publishes typed snapshots to Kafka, keyed by `asset_id`;
2. Spark Structured Streaming validates records, applies event-time watermarks and identifier deduplication, and maintains bounded per-asset state;
3. the processor pairs adjacent producer versions, normalizes the two states, computes a deterministic diff, evaluates configured rules, and emits diagnostics when state is incomplete;
4. the commit boundary writes authoritative history and derived evidence to MinIO/Parquet;
5. Elasticsearch, FastAPI, and the dashboard expose rebuildable metadata and read-only views for inspection.

3.2 State and pairing policy

The system consumes `config.snapshots` for core state reconstruction. It does not rely on the arrival order of `config.events`, and it does not require the `config.assets` topic to compute the core diff. For each asset, the processor keeps a bounded version buffer. After sorting by the producer-defined version field, it compares adjacent versions, not merely consecutive arrivals. A record that arrives late can therefore complete a transition that would be invalid under an arrival-order implementation.

The code stores the version in `ConfigSnapshot.version` as a producer-assigned integer. The notation v1, v2, and v3 below is an example, not a fixed dataset convention. For arrival order v1, v3, v2, the buffer first retains [v1,v3] and emits no adjacent edge. When v2 arrives within the retention policy, the sorted state is [v1,v2,v3] and the processor emits v1–v2 and v2–v3. It does not emit v1–v3 as an adjacent streaming transition. The pure comparison helper can accept non-consecutive endpoints and report a version gap; that separate API boundary must not be confused with the streaming emission rule.

The buffer is intentionally bounded. When a version gap, eviction, duplicate, or malformed record prevents a complete transition, the system records a diagnostic rather than inventing a diff. This choice makes incompleteness visible and keeps the correctness contract explicit. The default policy retains at most 32 versions per asset, applies a ten-minute event-time watermark delay, and clears inactive state after a 30-minute event-time timeout. The accepted TerraDS replay used its registered run settings, which must be reported separately from these defaults.

3.3 Common evidence contract

The common contract layer gives every record an identity, a time interpretation, and a provenance reference. A `ConfigSnapshot` carries `snapshot_id`, `asset_id`, producer version, event time, ingest time, and a raw content hash. Source-specific temporal records carry the corresponding source coordinate and manifest reference. A record identity is derived from canonical fields and a named policy; it is not inferred from a Kafka offset or from the order in which a consumer sees messages.

This distinction determines what the system can safely claim. A repeated delivery with the same logical identity can be suppressed or reconciled. A second record that reuses an identity with different content is a conflict. A timestamp identifies when an observation was made, but it does not establish which producer version precedes another. The processing code therefore keeps identity, producer order, event time, and transport provenance as separate fields.

3.4 Configuration transformation

Only an admitted adjacent pair enters the configuration transformation. The normalizer converts supported text into a deterministic line representation, applies the implemented whitespace and generated-comment rules, and masks selected secret-like settings. It produces a normalized hash while preserving the raw hash of the source bytes. These hashes answer different questions: the raw hash identifies the received content, while the normalized hash identifies the content under the declared policy.

The diff compares normalized lines with deterministic line-based logic. The result identifies the old and new snapshots, their versions, the policy, and the normalized hashes. YAML rules inspect the normalized diff and create deterministic alerts. The method supports the registered text representation. It is not a vendor-aware configuration parser, a complete secret detector, or proof that a proposed change is safe to deploy.

3.5 Source-specific temporal lanes

RPKI, RIS, and DNS do not pass through the configuration version buffer. They share the evidence contract and the publication boundary, but each lane has its own identity and completion condition.

RPKI. The RPKI path decodes source chunks into canonical authorization rows and records their source coordinates. A daily state is eligible for comparison only when the expected coordinate inventory is complete, the source manifest and artifact digest validate, and the accepted-row count and state digest agree with the observation descriptor. A transition compares two compatible consecutive complete states. An incomplete daily input remains incomplete; the processor does not infer deletions from rows that have not been observed.

RIS. The RIS path frames MRT input and projects records into peer-prefix occurrences. Occurrence identity retains the frame and record coordinate. Repeated observations are not collapsed into a set because multiplicity is part of the evidence. The resulting record answers an occurrence-processing question. It does not claim to be a complete global BGP state or a count of unique deployed routes.

DNS. The DNS path groups measurements by the declared query, response, address, and vantage dimensions. It carries coverage and vantage uncertainty into the output. A qualified

difference requires the declared comparison policy and the required partition or group completeness. An unknown coverage or vantage value remains unknown; the implementation does not turn it into evidence of global disappearance.

3.6 Publication and recovery

The publication boundary is lake-first. A batch writes raw provenance and typed processed records to Parquet, inventories the files, validates byte counts and row counts, computes or checks SHA-256 values, and writes a manifest only after the expected data is present. Readers use the manifest to exclude partial or uncommitted batches.

Search, FastAPI, and the dashboard are derived projections. If a projection is missing or stale, recovery reads the committed Parquet datasets, rebuilds the projection, and reconciles logical identities and counts. A failed projection does not change the authoritative history. The boundary does not make Kafka, Spark, MinIO, and Elasticsearch one distributed transaction, so the report does not claim end-to-end exactly-once processing.

3.7 Evaluation method

The evaluation follows the same boundaries as the implementation. Configuration replay checks snapshot identity, adjacent-pair identity, normalized output, alert identity, diagnostics, lake contents, and manifest integrity. RPKI checks complete-state membership, digests, transition equality, storage layout, and query equality. RIS checks framed occurrence identity and multiplicity. DNS checks qualified groups while retaining coverage and vantage fields.

Every performance result keeps its workload unit and processing boundary. TerraDS reports snapshots and adjacent pairs for an end-to-end replay. The matched B0/B2/B3 campaign compares one normalized projection under fresh processes and excludes source parsing and production state finalization. RPKI, RIS, and DNS rates are not pooled. This separation is part of the method because a row, an occurrence, a file version, and a measurement group do not represent the same object.

3.8 Normalization, diff, and publication

Normalization is deterministic and line-oriented for the supported configuration representation. The system retains a raw hash and a normalized hash, which lets the evaluator distinguish byte-level changes from semantic changes under the chosen normalization policy. Selected secret-like keys are masked according to the implemented rule set. The current method should not be described as a universal parser or a universal secret detector.

The publication rule is lake-first. A commit manifest records the relationship between authoritative Parquet artifacts and rebuildable projections. If Elasticsearch or the dashboard is unavailable, the historical record remains the recovery source. This design does not remove all failure modes, and it does not justify an end-to-end exactly-once claim. It does define where correctness must be checked first.

4 Current experimental evidence

4.1 Evidence roles and workload semantics

The evaluation uses distinct workloads for distinct questions. TerraDS represents a configuration-history replay with files and adjacent versions. RPKI, DNS, and RIS exercise processing paths with different record units and temporal structure. Their rates are not pooled into a single headline number.

Table 2: Accepted workload evidence at the checkpoint.

Workload	Main unit	Input size	Change/transition evidence	Interpretation
TerraDS	file/version	375 repositories, 10,188 versions	3,683 adjacent pairs and 12,278,085 logical bytes	End-to-end configuration-history replay
RPKI	canonical row	5,062,923 repeated tuples	12,016 additions/removals	Large row-oriented batch path
DNS	row/address group	2,234,863 rows	4,091 changes among 144,883 common groups	Grouped temporal comparison
RIS	peer-prefix occurrence	4,725,156 occurrences	Source characterization retained separately	Stream-like occurrence workload; full paced capacity remains pending

4.2 End-to-end TerraDS replay

The accepted TerraDS replay processed 10,188 snapshots and formed 3,683 adjacent diffs. It committed 29,402,997 raw and processed Parquet bytes, emitted zero alerts and zero diagnostics, and completed after the publisher drained in 68.742 seconds. The publisher rate was 500.023 snapshots/s. The accepted logical rate, which includes the workload's logical-byte interpretation, was 148.205 snapshots/s.

These numbers demonstrate an end-to-end path under the accepted replay conditions. They do not establish a production capacity limit. A supplemental headroom replay used the same history and pairs but produced 29,346,912 Parquet bytes and 139.565 accepted snapshots/s; because the runs were not a matched comparison, the difference should not be presented as a model improvement.

4.3 Corrected matched processing campaign

The corrected campaign contains 27 exact trials: nine configurations with three fresh-process repetitions each. The compared B0, B2, and B3 modes produced identical full normalized rows in all accepted trials. B3 also reported zero selected-offset lag. The table reports medians with observed ranges in seconds.

The matched result supports a narrow conclusion: under the tested normalized-core workload,

Table 3: Matched processing medians and observed ranges.

Workload	B0	B2	B3
RPKI (360,226 rows)	6.297 (6.282–12.577)	16.840 (14.729–21.080)	25.417 (23.144–25.424)
RIS (370,141 occurrences)	10.485 (8.437–12.636)	21.167 (18.967–23.186)	29.644 (23.233–29.726)
DNS (4,096 rows)	2.089 (2.087–6.302)	10.536 (10.486–12.584)	16.866 (12.624–16.924)

the modes agree on output while incurring different processing times. The result excludes production-specific parsing, state finalization, and full production ranking. It is therefore a component comparison, not a claim that one mode is universally superior.

4.4 Storage and resource observations

The matched campaign produced 1,622,766,994 Parquet bytes and 1,623,023,143 MinIO namespace bytes. Retained growth was 3,869,485,576 bytes, below the amended 6 GiB limit. The peak tree was 1,612,824,576 bytes, the minimum observed guest RAM was 4,191,440,896 bytes, and no resource failure occurred. The aggregate processing time was 502.981 seconds, below the 7,200-second acceptance limit.

The initial pilot failed because a SQLite connection was used from the wrong Spark callback thread. That attempt produced zero committed files and is retained as a failed trial. The corrected retry passed. Reporting both attempts matters because the failure identifies a real integration boundary rather than an unexplained missing result.

5 What is complete and what remains

5.1 Completed at the checkpoint

- The core data contracts, topic boundaries, and asset-keyed ingestion path are defined.
- The streaming processor implements event-time buffering, deduplication, adjacent-version pairing, deterministic normalization, diffing, alert evaluation, and diagnostics.
- MinIO/Parquet provides the authoritative historical path, while the search/API layer is treated as a rebuildable projection.
- A Kubernetes single-node acceptance run covers the real Kafka-to-Spark-to-MinIO path for the current deployment scope.
- The read-only dashboard exposes historical and experimental evidence rather than mutable operational controls.
- TerraDS replay and the corrected matched campaign provide auditable evidence with explicit acceptance boundaries.

5.2 Not yet complete

The project has not completed a full paced RIS capacity comparison, the H3 branch, a multi-node high-availability deployment, or a failure campaign that establishes recovery guarantees under realistic component loss. The current artifacts also do not justify an end-to-end exactly-once claim. These are evaluation gaps, not implementation details that can be silently inferred from the successful replays.

The paper still needs a final human pass over hardware and operating-system metadata, reference formatting, author contributions, data/code availability, and the exact relationship between each table and its source artifact. Those items are part of the path from an accepted checkpoint to a submission-ready report.

6 Risk assessment

Table 4: Main risks and mitigation actions.

Risk	Why it matters	Mitigation for the next checkpoint
Overclaiming scalability	Single-node acceptance cannot represent cluster capacity or HA.	Report deployment scope explicitly and run a matched multi-node or paced-load study if the claim is required.
Mixed workload units	Rows, occurrences, files, and bytes answer different questions.	Keep a workload-role matrix and report denominators next to every rate.
Projection drift	Search results can diverge from authoritative history.	Add manifest-based reconciliation and a documented rebuild test.
Late or missing state	A bounded buffer can evict a version before its predecessor arrives.	Expand adversarial out-of-order tests and publish diagnostic counts by reason.
Reproducibility gaps	A result without environment and artifact provenance is hard to audit.	Freeze run manifests, hardware metadata, seeds, and exact command lines.

7 Next steps

The next phase should reduce uncertainty rather than add more components. The recommended order is:

1. **Close the evidence ledger.** Attach every reported number to a run manifest, environment record, artifact path, and acceptance criterion. Record the exact hardware and operating-system details.
2. **Finish the deferred comparisons.** Run the approved paced RIS experiment and H3 only if their hypotheses and denominators are fixed in advance. Keep them separate from

the accepted matched campaign.

3. **Test recovery and projection rebuild.** Delete or make unavailable a rebuildable projection, reconstruct it from Parquet and the manifest, and verify row counts, hashes, and query results. Then document the limits of the recovery guarantee.
4. **Expand correctness tests.** Add adversarial cases for duplicate identifiers, late versions, version gaps, buffer eviction, malformed records, and restart boundaries. Check both positive diffs and diagnostics.
5. **Prepare the final presentation.** Use one architecture figure, one state-reconstruction example, one evaluation table, one evidence dashboard view, and one limitations/next-steps slide. Avoid a service inventory as the main story.

Table 5: Milestones after the 25 October 2026 evidence boundary.

Milestone	Deliverable	Exit condition
M1	Reproducible evidence pack	Every accepted result has a manifest, environment, command, output artifact, and reviewer-readable interpretation.
M2	Deferred evaluation and recovery	RIS/H3 decisions are documented; projection rebuild and adversarial state tests pass or are explicitly bounded.
M3	Final paper and talk	Claims, figures, tables, references, and speaker narrative agree with the evidence ledger.

8 Conclusion

At the checkpoint, ConfigStream is a functioning evidence-oriented prototype with a substantially complete implementation path and accepted replay results. Its strongest current contribution is the separation of temporal state reconstruction, authoritative history, and rebuildable read projections. Its strongest limitation is the narrow deployment and evaluation envelope. The next report should therefore focus on closing evidence gaps and tightening claims, not on presenting the current single-node results as a general scalability result.